UNSW AUSTRALIA

2026 Thesis Project

Never Stand Still | Faculty of Engineering | School of Electrical Engineering and Telecommunications

Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

Author: Jordan Mohindra

Supervisor: A/Prof Beena Ahmed

Why compress radar data?

- Cascaded MIMO radars (here **12 transmitters × 16 receivers**) produce **3.1 MB of raw data in every frame**, frame after frame.
- The link to the car's central computer (SoC) becomes the bottleneck: bandwidth, power and delay.
- So: **compress at the sensor, losslessly**. Less data on the link, **identical** data after decoding.

Radar 393 KB / frame → **FPGA** compress → **93 KB** → **SoC** decompress

same data, bit for bit

Background and aim

- There are many ways to compress radar data, and **no single best one**: each trades **compression** against **accuracy**, **speed** and **hardware cost**.
- In a car, lost detail can mean a **missed pedestrian**, so a method has to be judged on whether **targets survive**, not just on how small the data gets.
- Results **depend on the data**: a method that does well on simulated noise can behave differently on **real radar recordings**.
- And only **real hardware** shows whether a method is **fast and small enough** to run at the sensor.

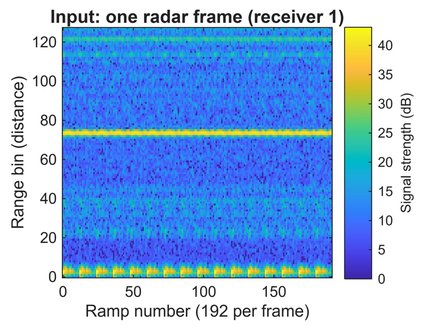
Aim: compare compression methods side by side on **real radar data**, pick the best **lossless** ones, and build the **encoder and decoder on an FPGA**.

Method → Results

1 Real radar data

Recordings from the **ColoRadar** dataset [1], captured with a **TI AWR2243 cascade** radar (12 TX × 16 RX).

In MATLAB each frame gets a **range FFT** and is stored as **16-bit integers**: the input to every method.



Data	Details	Size
Dataset	**ColoRadar** [1]: real radar recordings, open source	–
Radar	TI MMWCAS-RF-EVM: 4 × AWR2243 chips, **12 TX × 16 RX**	–
Frames used	**50 frames**, each processed on its own	19.7 MB
One frame	128 range bins × 192 ramps × 4 RX, 16-bit I/Q (after range FFT)	**393 KB**

Each method is judged on:

$$\mathrm{CR} = \frac{\text{input bits}}{\text{compressed bits}}$$

Missed targets: targets found in the original data but **not** after compression.

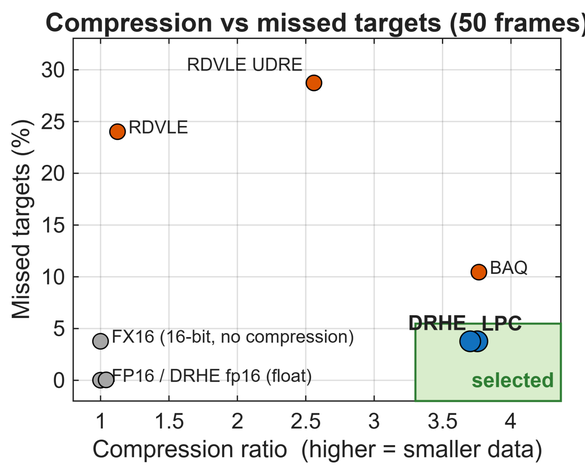
2 Screen 8 methods

Every method is run in **MATLAB** on the same **50 frames**.

Each is scored on **compression ratio** and **missed targets**: targets the detector finds in the original data but not after compression.

Lossy methods (BAQ, RDVLE) shrink the data but **lose targets**. Stored as **float**, prediction barely works (CR ≈ 1).

Only **DRHE and LPC** are lossless **and** above 3×, so they go on to hardware.



Method	How it works	CR	Missed
FP16	16-bit float, uncompressed	1.00	0.0 %
FX16	16-bit integer, uncompressed	1.00	3.8 %
BAQ	Block adaptive quantisation (lossy)	3.76	**10.4 %**
RDVLE	Vector quantisation (lossy)	1.12	**24.0 %**
RDVLE UDRE	Vector quantisation variant (lossy)	2.56	**28.7 %**
DRHE fp16	DRHE on float data	1.04	0.1 %
DRHE	Predict magnitude & phase, Huffman	**3.75**	**3.8 %**
LPC	Linear prediction, Huffman	**3.70**	**3.8 %**

Green = selected: high CR **and** no extra missed targets. The 3.8 % comes from 16-bit rounding, not compression.

What are DRHE and LPC?

Both **predict** each sample x_m from earlier ramps of the **same transmitter** (L = 12 back) and **Huffman-code the error**:

$$e_m = x_m - \hat{x}_m$$

Huffman coding: the most common error sizes get the **shortest codes**, from a **dictionary** re-derived for this data.

DRHE predicts **magnitude A** and **phase φ** with **fixed** filters:

$$\hat{x}_m = \hat{A}_m\, e^{j\hat{\phi}_m} \qquad \hat{A}_m = \alpha\, \hat{A}_{m-L} + (1-\alpha)\, A_{m-L}$$

Past ramps only → **streams**

LPC takes a **weighted sum** of the previous two ramps:

$$\hat{x}_m = a_1\, x_{m-L} + a_2\, x_{m-2L}$$

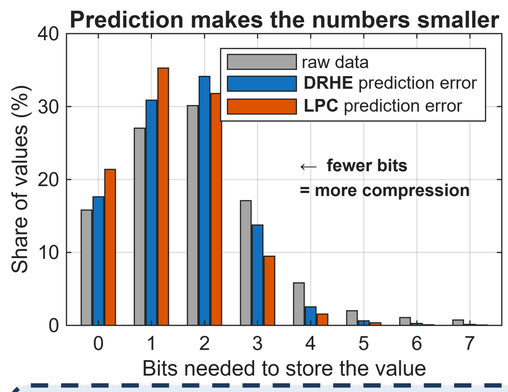
a_1, a_2 **fitted to the whole frame** → waits

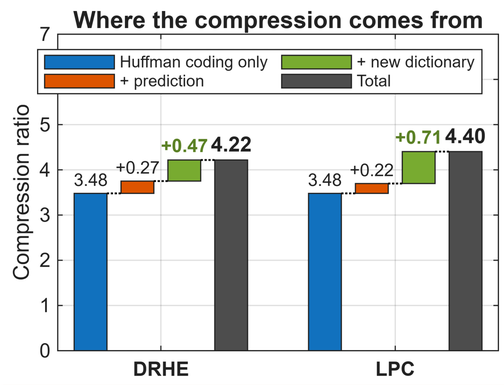
3 DRHE vs LPC

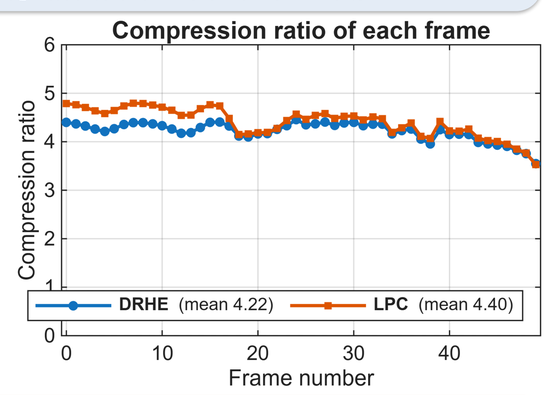
Both are run on the **FPGA test set** (4 RX, 50 frames).

The compression is split into three parts: **Huffman coding** alone, **+ prediction**, and **+ a dictionary re-derived** from the first 5 frames.

The new dictionary adds more than the predictor.







4 Build it in hardware

The MATLAB model is rewritten in **C++**, turned into hardware with **Vitis HLS**, and built with **Vivado** for the AMD **KCU116** board.

Encoder and decoder both run on the chip.

[Photo: KCU116 FPGA board and laptop running the test]

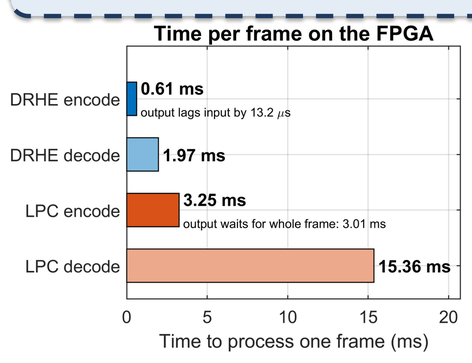
MATLAB model → **C++** Vitis HLS → **Vivado** bitstream → **KCU116** FPGA board

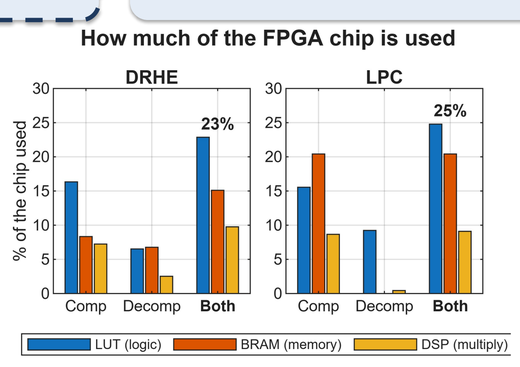
The **same C++** that was simulated is built for the chip, so the board's output can be checked **bit for bit** against the simulation. Encoder and decoder each run as their own core, fed by DMA from the board's memory.

5 Measure on the board

All 50 frames are streamed through the FPGA by **DMA**.

Every output is checked **bit for bit** against the simulation, and the **time per frame**, **delay** and **chip use** are measured.





On the FPGA	DRHE	LPC
Frames bit-exact	**50 / 50**	**50 / 50**
Compression ratio	4.22×	**4.40×**
Encode time / frame	**0.61 ms**	3.25 ms
Output starts	**as data arrives**	after whole frame
Chip logic (LUTs)	**23 %**	25 %
Decode time / frame	1.97 ms	15.36 ms

Why an FPGA? A proof of concept

- The FPGA is a **proof of concept**: it shows the compressor works as **real hardware**, not just as MATLAB code.
- It measures what simulation cannot: **speed, delay and chip size**, with output **bit-exact** to the simulation.
- In a production car the design would move to a **custom chip (ASIC)** or a **block inside the radar SoC**, which is cheaper and uses less power.

Conclusion

- **Both DRHE and LPC are lossless on real radar data, on real hardware.**
- **LPC compresses most: 4.40×** (DRHE 4.22×). A **re-derived Huffman dictionary** helped **more than prediction** did.
- **Delay:** DRHE's output lags its input by only **13 µs**; LPC's waits for the **whole frame (3.0 ms)**. Decoding: **1.97 ms** (DRHE), 15.4 ms (LPC).
- **DRHE streams** with no frame buffer, so it **scales better to bigger frames**.

[1] A. Kramer et al., ColoRadar: the direct 3D millimeter wave radar dataset, IJRR, 2022.